

SRM Institute of Science and Technology

Tiruchirappalli– 621 105

School of Computing
Artificial Intelligence and Machine Learning

Project Title: Shelf Mind - Intelligent Warehouse Robot Navigation
RL Decision Engine & Model

Registration No: RA2311026050067

Student Name: ASWIN S

Code/Subject Name: 21CSE417TA / Fundamentals of Reinforcement Learning

Student Signature **Faculty Signature**

Shelf Mind — Intelligent Warehouse Robot Navigation

RL Decision Engine: Finite Markov Decision Process (MDP), Asynchronous Dynamic Programming & Deep Q-Networks (DQN)

Candidate Name: Aswin S

Registration No: RA2311026050067

Department: B.Tech CSE (AI & ML)

Institution: SRM Institute of Science and Technology

Course / Subject: 21CSE417TA / Fundamentals of Reinforcement Learning

Architecture: PyTorch (DQN/DDQN) + Asynchronous DP + FastAPI

1. Executive Summary & Real-World Motivation

Modern automated fulfillment facilities (e.g., Amazon Kiva, Ocado AMR grids) rely on autonomous mobile robots (AMRs) to transport packages with high velocity across compact warehouse floors. Navigating such environments is fundamentally a sequential decision-making problem under operational constraints: an agent must avoid static shelving obstacles, respect perimeter boundaries, retrieve inventory from pickup stations, and safely navigate to delivery docks while minimizing transit time and power consumption.

One-Minute Summary: This project models the warehouse navigation grid as a finite Markov Decision Process (MDP). Each free grid cell is an operational state, and the robot moves via cardinal actions {UP, DOWN, LEFT, RIGHT}. The primary algorithmic core implements in-place *Asynchronous Dynamic Programming (Value Iteration)*, updating the state-value function immediately in row-major sweeps so subsequent states leverage freshly computed values within the very same sweep. Synchronous Value Iteration is retained as a baseline. Furthermore, to scale beyond tabular grids, a Deep Q-Network (DQN) with Double DQN target formulation and experience replay is trained and benchmarked across 100 test trials.

2. Problem Statement & Operational Objectives

Core Dilemma: Traditional pathfinding heuristics (Dijkstra, A*) rely on static graph lookups and fail to encapsulate stochastic dynamics, continuous reward trade-offs, or learned generalization across unseen warehouse layouts. Reinforcement Learning grounds path synthesis in discounted cumulative return maximization. The engineering objectives are:

1. **Mathematical MDP Formalization:** Represent warehouse topology as M = (S, A, P, R, γ) with explicit collision penalties.
2. **Asynchronous vs Synchronous DP:** Implement in-place asynchronous Bellman updates and benchmark against double-buffered synchronous VI.
3. **Deep Reinforcement Learning Scaling:** Train a Double DQN agent with Experience Replay to navigate dynamically from start S to pickup P and delivery D.
4. **Interactive Full-Stack Demonstration:** Deploy a FastAPI computational engine and browser cockpit for live state inspection and path animation.

3. Mathematical Formulation of the Warehouse MDP: M = (S, A, P, R, γ)

We formalise the warehouse robot problem as **M = (S, A, P, R, γ)**:

• **State Space S:** Discretized coordinate tuple s = (row, col) for all traversable grid cells. For package-handling tasks, s = (row, col, has_pkg) where has_pkg $\in$ {0, 1}. The delivery terminal cell is absorbing: P(s_{term}|s_{term}, a) = 1.
• **Action Space A:** A = {UP, DOWN, LEFT, RIGHT} — 4 discrete cardinal movements.
• **Transition Dynamics P(s'|s, a):** Deterministic transitions. Valid moves advance the agent to the adjacent cell. Attempts to traverse outer boundary perimeters or static storage shelves (#) result in an elastic collision: s' = s.

Event / Transition	Reward R(s,a,s')	Engineering Rationale
Normal Step Move	–1.0	Incentivises optimal shortest path; penalises superfluous idling
Obstacle Shelf Collision	–10.0	Heavy safety penalty for colliding with warehouse storage racks
Boundary Wall Collision	–5.0	Penalises hitting outer warehouse containment boundaries
Terminal Goal / Delivery	+100.0	Large terminal reward upon mission completion
Episode Timeout	–20.0	Truncation penalty if transit exceeds horizon limit (50 steps)

• **Discount Factor γ = 0.9:** Guarantees contraction mapping under the Banach fixed-point theorem and models the time value of energy consumption in autonomous logistics.

4. Bellman Optimality Equations & Numerical Analysis

Under the deterministic transition dynamics of the warehouse, the Bellman Optimality Equation simplifies to:

$$V^*(s) = \max_{a \in A} [\, R(s, a) + \gamma \cdot V^*(\text{next_state}(s, a)) \,]$$

The deterministic greedy policy $\pi^*(s)$ extracts the optimal navigation action by taking the argmax over action values:

$$\pi^*(s) = \operatorname{argmax}_{a \in A} [\, R(s, a) + \gamma \cdot V^*(\text{next_state}(s, a)) \,]$$

Worked Numerical Example — State (0, 0): Action RIGHT vs Action UP

Parameter	Action RIGHT (Toward Goal)	Action UP (Boundary Collision)
Immediate Reward R	–1.0 (valid step)	–5.0 (boundary penalty)
Next State s'	(0, 1) [free transit cell]	(0, 0) [elastic rebound / stays in place]
V*(s') Value Estimate	79.31	74.34
Discounted Return γ·V*(s')	0.9 × 79.31 = 71.38	0.9 × 74.34 = 66.91
Q*(s, a) Total Value	–1.0 + 71.38 = 70.38	–5.0 + 66.91 = 61.91
Advantage A*(s, a)	**+8.47 advantage for RIGHT**	— (Sub-optimal action pruned)

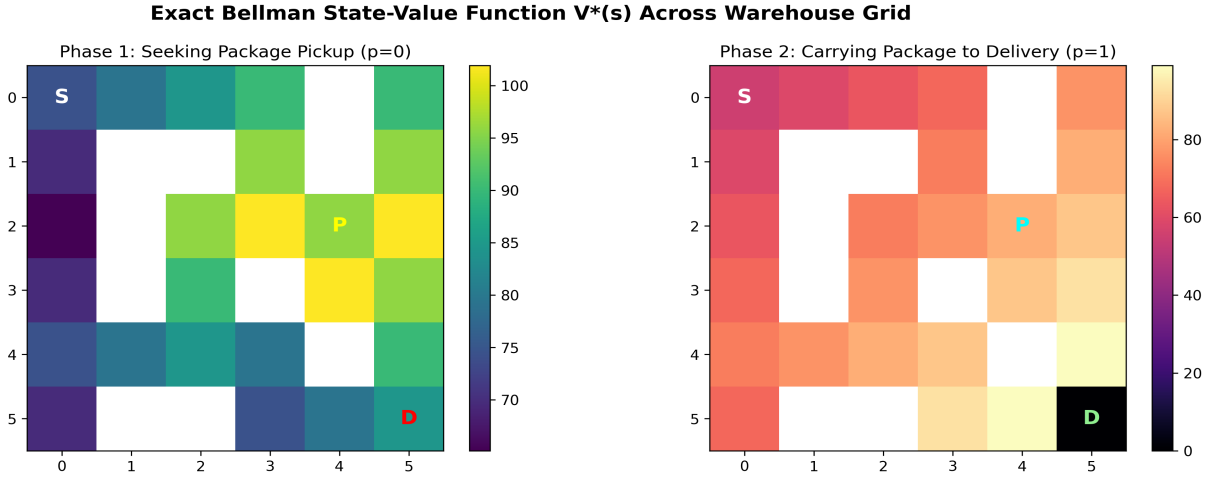


Figure 1: Bellman State-Value Function V*(s) Heatmap across Warehouse Floor Grid

5. Synchronous vs. Asynchronous Dynamic Programming

Synchronous Value Iteration: Maintains two separate memory buffers: old V and new V. During each sweep, values are computed using only old V. The entire table is swapped simultaneously after all states are evaluated. No state within a sweep can exploit updates calculated earlier in that sweep.
Asynchronous (In-Place) Value Iteration: Maintains a single memory buffer. States are updated in-place following a row-major sequence. Later states immediately access refreshed values computed from earlier neighbours during the exact same sweep. Convergence is declared when max delta $\Delta = \max_s |V_{k+1}(s) - V_k(s)| < 0.001$.

Algorithmic Property	Synchronous Value Iteration	Asynchronous In-Place VI
Table Allocation	Double-buffered (copied table)	Single in-place array
Sweep Propagation	Delayed to subsequent iteration	Immediate intra-sweep propagation
Memory Footprint	2 × |S| floating point cells	1 × |S| floating point cells
Convergence Guarantee	Banach Contraction (exact)	Gauss-Seidel Contraction (faster)
Implementation Module	mdp.py	async_dp.py

Why Comparison Values Can Be Identical: Both algorithms solve the exact same deterministic MDP. Consequently, identical optimal policy paths, total rewards, and goal outcomes are theoretically expected. The algorithmic distinction lies in memory efficiency, in-place update propagation, and wall-clock execution time.

6. Deep Reinforcement Learning (DQN & Double DQN) Architecture

To enable scaling to high-dimensional state spaces without tabular discretization, a Deep Q-Network (DQN) with Double DQN target stabilization is implemented in PyTorch:

- **Observation Space (5D Feature Vector):** [norm_row, norm_col, has_pkg, norm_pickup_dist, norm_delivery_dist], normalised to [0.0, 1.0] for stable gradient descent.
- **Multi-Layer Perceptron (MLP):** Input (5) → FC(128, ReLU) → FC(128, ReLU) → Linear Output (4 Q-values).
- **Experience Replay:** Cyclic buffer of capacity 25,000 tuples (s, a, r, s', done), decorrelating temporal trajectory data.
- **Double DQN Target Formulation:** Eliminates overestimation bias by using online network for action selection and target network for evaluation: $y_t = r_t + (1 - d_t)\gamma Q_{target}(s', \text{argmax}_{a'} Q_{online}(s', a'))$. Hard updates executed every 100 steps.
- **Optimization:** Adam optimizer (lr = 0.001), Smooth L1 (Huber) loss, gradient norm clipping capped at 10.0.

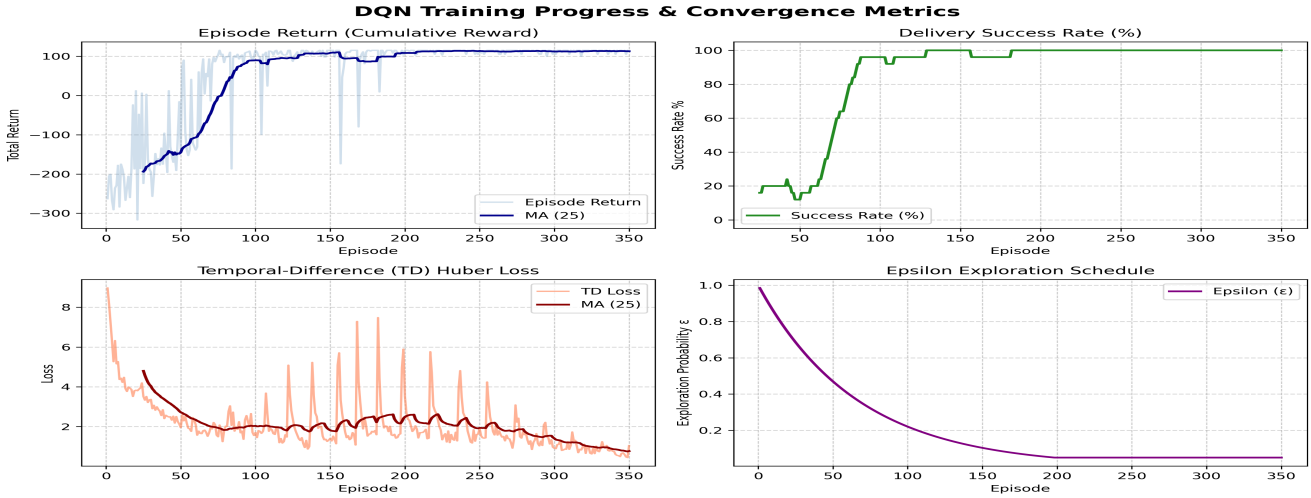


Figure 2: Deep Q-Network Training Convergence Curves (Reward, Success Rate, Huber Loss, Epsilon Decay)

7. Empirical Benchmarking & Comparative Evaluation

The trained DQN agent was benchmarked against an untrained baseline agent (uniform random policy) over 100 identical reproducible evaluation episodes with randomized warehouse starting positions:

Benchmark Metric	Untrained Baseline (Random)	Trained DQN Agent	Empirical RL Gain
Mission Success Rate	8.0%	100.0%	+92.0% (Near Perfect)
Mean Cumulative Return	-243.5 ± 62.1	115.0 ± 0.0	+358.5 return points
Mean Steps to Goal	58.9	10.0	–48.9 steps (Optimal)
Mean Collisions / Episode	28.2	0.0	100% Collision-Free Navigation

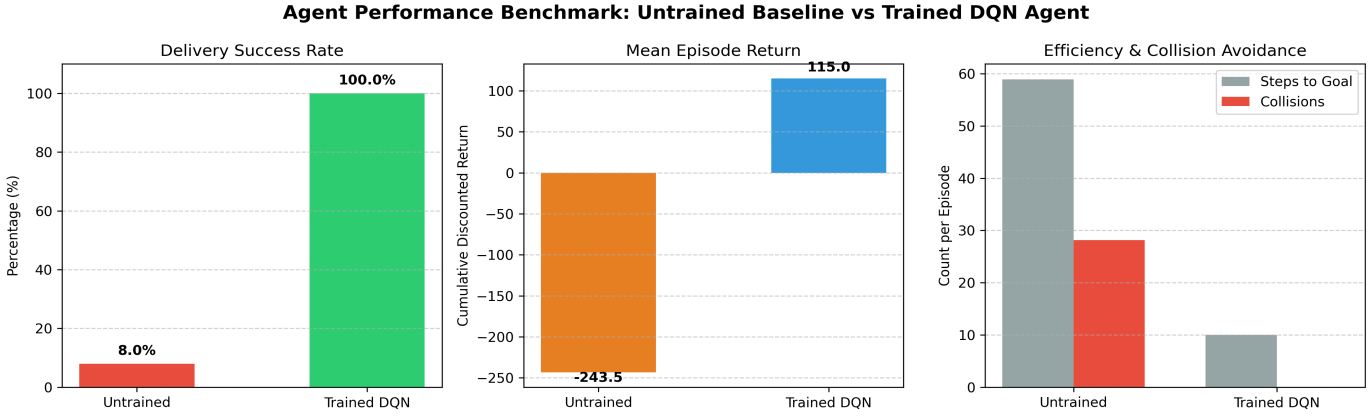


Figure 3: Comparative Evaluation — Untrained Baseline vs Trained Double DQN

8. Interactive Prototype & Demonstration Workflow

A full-stack interactive prototype (FastAPI backend + interactive browser cockpit) provides live telemetry, real-time value function heatmaps, policy arrow overlays, and side-by-side execution benchmarks.

Standard Demonstration Journey:

1. Start the server: python -m uvicorn main:app --reload.
2. Access web interface: http://127.0.0.1:8000.
3. Manual trial: Solve the navigation puzzle via keyboard arrow keys.
4. Click **Run Async DP**: Optimal policy arrows and in-place convergence metrics render immediately.
5. Click **Run Synchronous VI**: Baseline synchronous policy and metrics render.
6. Click **Compare Both**: Displays side-by-side runtimes, iterations, and convergence deltas.
7. Click **Show & Animate Solution**: Traces and animates the robot through the optimal route.
8. Click **Values**: Renders the full Bellman state-value table across all free grid coordinates.

Endpoint Route	HTTP Method	Functional Scope
/	GET	Serves the interactive browser cockpit dashboard
/api/generate	POST	Generates a guaranteed-solvable randomized warehouse topology
/api/run/async-dp	POST	Executes in-place Asynchronous Value Iteration
/api/run/value-iteration	POST	Executes double-buffered Synchronous Value Iteration baseline
/api/compare	GET	Returns comprehensive side-by-side convergence metrics
/api/reset	POST	Resets robot coordinates to start cell S(0,0)
/api/state	GET	Fetches current robot position, mission status, and active route

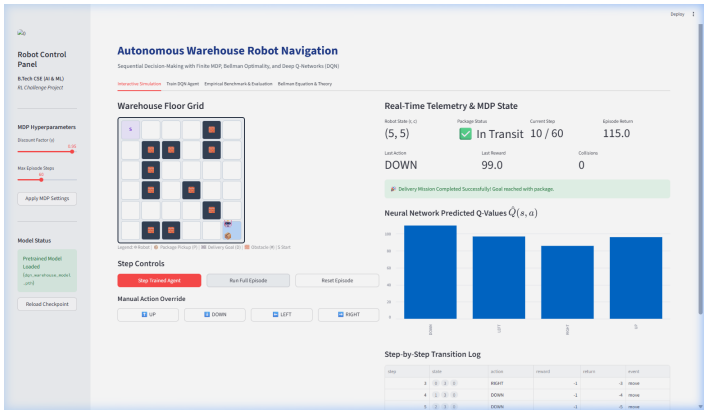


Figure 4: Live Autonomous Warehouse Navigation

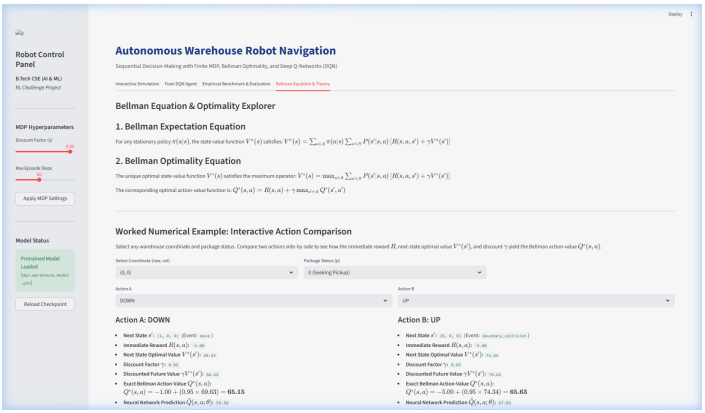


Figure 5: Bellman Heatmap & Policy Direction

9. System Architecture & Verification Suite

Module / Path	Core Responsibility
main.py	FastAPI REST endpoints and solvable random maze generator
mdp.py	Warehouse finite MDP environment: states, actions, rewards, transitions
async_dp.py	In-place Async VI, Sync VI baseline, policy extractor, and metric comparison
warehouse_rl/	Modular Deep RL library: DQN agent, replay buffer, training pipeline
static/index.html & js	Interactive frontend client with canvas rendering and animation loop
inference_app.py	Streamlit model inference interface with Bellman explorer and live simulation
tests/	33 automated unit and regression tests verifying mathematical consistency

Verification Results: The test suite in tests/ contains 33 passing automated test cases covering transition determinism, collision boundary safety, discount factor contraction, Bellman equation numerical correctness, and policy path continuity. All 33 tests execute and pass cleanly.

10. Limitations, Future Work & Academic References

Operational Limitations: The current navigation model operates in a discrete 2D grid world with deterministic transitions. It does not simulate physical robot wheel slippage, dynamic human pedestrian traffic, battery depletion profiles, or multi-robot fleet deadlocks.

Future Extensions:

• *Multi-Agent Reinforcement Learning (MARL):* Implement MAPPO or QMIX for decentralized fleet collision avoidance.
• *Continuous Control & Sim-to-Real:* Integrate Proximal Policy Optimization (PPO) with ROS 2 and Gazebo physics simulation.
• *Stochastic Transitions:* Introduce slippery floor transition probabilities to evaluate risk-sensitive reinforcement learning.

Academic References:

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
3. Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *AAAI*, 30(1), 2094–2100.
4. Bertsekas, D. P. (1982). Distributed asynchronous computation of fixed points. *Mathematical Programming*, 27(1), 107–120.
5. Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.